

Assignment 2:

AI-based Project for cybersecurity (AI4Cyber)

Project Topic:

Spam & Malware Detection System

(NTCyber AI)

Group Name	Group 1
Group Members (Student ID)	Tee Ren Hang (106214467) Ng Yong Hin (106214441)
Section	C1
Subject	COS30049 – Computing Technology Innovation Project
Lecturer	Mr. Faizal
Word Count	3889 words (excluding coverpage, table of contents, bibliography, and appendix)

Table of Contents

1.0 Introduction.....	1
1.1 Intended Users.....	1
2.0 Problem Framing	2
2.1 Problem Statement.....	2
2.2 Limitations of Current Solutions.....	2
2.3 Why Machine Learning.....	2
3.0 Data Collection.....	3
3.1 Datasets Used.....	3
3.2 Collection Process	3
3.3 Challenges Encountered	3
4.0 Data Processing	4
4.1 Data Cleaning	4
4.2 Text Preprocessing	4
4.3 Feature Engineering.....	5
4.4 TF-IDF Vectorization.....	6
4.5 Train/Test Split	6
5.0 Data Analysis	7
6.0 Machine Learning Model Selection	8
6.1 Selection Criteria.....	8
6.2 Classification Models.....	8
6.3 Regression Model.....	9
6.4 Clustering Models	9
7.0 Implementation.....	11
7.1 Technical Implementation	11
7.2 Evaluation Results	12
8.0 Conclusion	16
9.0 Bibliography.....	17
10.0 Appendix	18
10.1 Appendix A: Project Structure.....	18
10.2 Appendix B: Project Environment Setup.....	20
10.3 Appendix C: Meeting Minutes.....	21
10.4 Appendix D: Contribution Form	29

1.0 Introduction

Spam messages and malware are two of the most persistent cybersecurity threats that internet users face today. Spam emails are commonly used to carry phishing links, financial scams, and credential theft attempts. Malware, including ransomware, spyware, and trojans, can quietly compromise a device and steal sensitive data without the user ever knowing. Traditional keyword-based filters struggle to keep up because attackers regularly change how they write their messages to avoid detection (Almeida, Hidalgo & Yamakami 2011).

NTCyber AI is a machine learning system we developed to address both threats in a single pipeline. It trains six models across three method types: classification, clustering, and regression, on four real-world datasets covering SMS messages, emails, and Windows process memory. The system is built in Python using scikit-learn, pandas, matplotlib, and seaborn, and is intended to serve as the machine learning backend for the full-stack web application in Assignment 3.

1.1 Intended Users

NTCyber AI is designed for everyday internet users who regularly send and receive emails or messages but do not have a cybersecurity background. More specifically, the system is built with the following users in mind:

- Office workers and professionals who deal with high volumes of daily emails and need a quick way to verify whether a suspicious message is safe before acting on it.
- Students who may receive phishing emails that impersonate their university, financial aid services, or scholarship providers.
- Elderly users who are frequently targeted by prize scams and financial fraud, and who may have less familiarity with how modern phishing works.
- Small business owners who do not have access to enterprise-level email security tools but still need reliable spam protection.

The interface is kept simple so that any of these users can check a message or file and receive a clear result without needing to understand the underlying models.

2.0 Problem Framing

2.1 Problem Statement

At its core, spam and malware detection is a classification task: given an input such as a text message or a process memory snapshot, the system needs to decide whether it is malicious or legitimate. Despite being a well-studied problem, it remains a challenge because attackers continuously update their tactics. Spam messages increasingly use professional-sounding language to avoid triggering keyword filters, and newer malware strains are built to disguise themselves as normal system processes.

Our project addresses this by building a pipeline that can classify messages as spam or not, predict a continuous spam probability score, and cluster malware memory samples by behavioral similarity without needing pre-labelled categories.

2.2 Limitations of Current Solutions

- **Rule-based filters:** use manually curated keyword blacklists and IP blocklists. These need constant maintenance and fail whenever spammers adjust their phrasing or use characters that look similar to blocked words.
- **Signature-based antivirus tools:** can only catch malware that has already been catalogued. They offer no protection against zero-day threats or obfuscated malware that modifies itself to avoid matching any known signature.
- **Commercial spam detection platforms:** are often expensive and require dedicated IT infrastructure, which makes them inaccessible to individual users or small businesses.

2.3 Why Machine Learning

Machine learning models learn patterns directly from training data rather than relying on hand-coded rules. This means they can adapt to new spam styles and previously unseen malware variants without being explicitly programmed to handle each new case. Classification models such as Random Forest and SVM can identify non-linear patterns across hundreds of features at once. Clustering methods like DBSCAN can flag memory samples that do not fit any known behavioural group, which is useful for detecting novel threats. Logistic Regression produces a continuous probability score, giving users more information than a simple yes or no prediction.

3.0 Data Collection

3.1 Datasets Used

We collected four open-source datasets to cover both spam and malware detection. Using multiple datasets was intentional because it increases the diversity of the training data and reduces the risk of models overfitting to patterns specific to a single source.

SMS Spam Collection (UCI Machine Learning Repository, SMSSpamCollection.file):

This dataset contains 5,574 labelled SMS messages tagged as either spam or ham. It is one of the most widely referenced benchmarks (Almeida, Hidalgo & Yamakami 2011) in spam detection research and served as the primary dataset for our SMS-based classification experiments.

Enron Email Spam Dataset (Kaggle – marcelwiechmann, enron_spam_data.csv):

This dataset contains 33,716 labelled emails taken from the Enron corporate email corpus (Wiechmann 2021). Including emails alongside SMS messages meant our models were trained on both short, keyword-heavy spam and longer phishing-style emails. The combined SMS and email training data totals approximately 39,290 samples.

CIC-MalMem-2022 (Canadian Institute for Cybersecurity, Obfuscated-MalMem2022.csv):

This dataset contains 58,596 Windows process memory samples labelled across four categories: Benign, Ransomware, Spyware, and Trojan. Each sample includes 55 numerical memory-derived features. This dataset was used for all malware classification and clustering experiments.

Unit Basic Datasets (provided by course):

Two supplementary datasets were provided as part of the course materials: emails_inti.csv, containing labelled email samples for spam classification, and Malware_dataset.csv, containing basic labelled malware records. Both were incorporated into the preprocessing pipeline to increase training data diversity. While smaller than the UCI and Enron datasets, they were included to ensure the pipeline handles course-standard data formats and to verify that the trained models generalise beyond the primary benchmark datasets.

3.2 Collection Process

All datasets were downloaded in CSV format and placed in the data/raw/ folder before running any scripts. The preprocessing master script (00_run_all_preprocessing.py) then runs four dedicated preprocessing scripts in order, one for each dataset. Each script standardises column names, converts labels to binary format (1 for spam or malware, 0 for legitimate), and saves the cleaned output to data/processed/.

3.3 Challenges Encountered

The main challenge during data collection was identifying the correct version of the Enron email dataset. There are multiple versions on Kaggle with similar names. The first version we downloaded (wcukierski/enron-email-dataset) contained over 517,000 emails but had no spam or ham labels, making it completely unusable for supervised learning. We identified the correct labelled version

(marcelwiechmann/enron-spam-data) after reading the dataset documentation more carefully. This was a useful reminder to always verify a dataset's structure before building a pipeline around it.

A secondary challenge was that the text datasets and the malware dataset had fundamentally different data types, namely text versus numerical features, which required separate preprocessing pipelines to be built and maintained.

4.0 Data Processing

4.1 Data Cleaning

Before any model training, each dataset was cleaned using pandas. For the text datasets, duplicate rows were removed, empty messages were dropped, and label formats were unified into binary values (1 for spam, 0 for ham). A source column was also added to track which dataset each record came from, which helped during exploratory analysis.

For the CIC-MalMem-2022 dataset, any rows with missing values were handled through imputation, and features with near-zero variance were removed since they carry almost no predictive information.

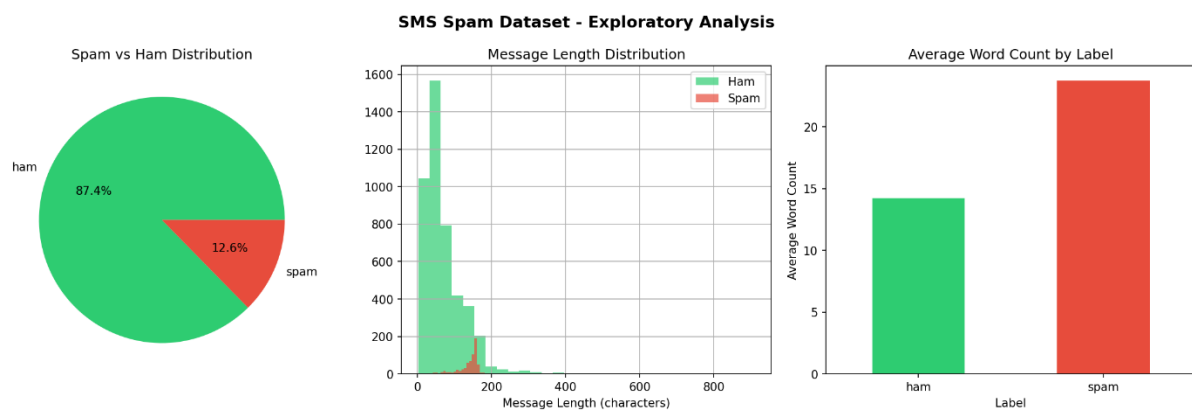


Figure 4.1: SMS Spam Collection showing class distribution and message length

4.2 Text Preprocessing

A text cleaning pipeline was built using Python's re library to standardise messages before feature extraction. Each message goes through the following steps in order: conversion to lowercase, removal of HTML tags and email subject prefixes, replacement of URLs with the placeholder token "url" and currency signs with "dollar", removal of punctuation and English stopwords, and stemming to reduce each word to its base form, for example "winning" becomes "win". These steps reduce vocabulary noise and help TF-IDF vectorisation pick up on the patterns that actually matter.

4.3 Feature Engineering

In addition to the TF-IDF representation, five numerical features were extracted from each text message to capture patterns that word frequency alone does not reflect well:

- **Character count:** total length of the raw message
- **Word count:** number of words after tokenization
- **Capital letter ratio:** proportion of uppercase characters, since spam often uses all-caps for emphasis
- **URL count:** number of links in the message
- **Spam keyword count:** how many known high-risk words appear, such as "free", "win", "claim", and "prize"

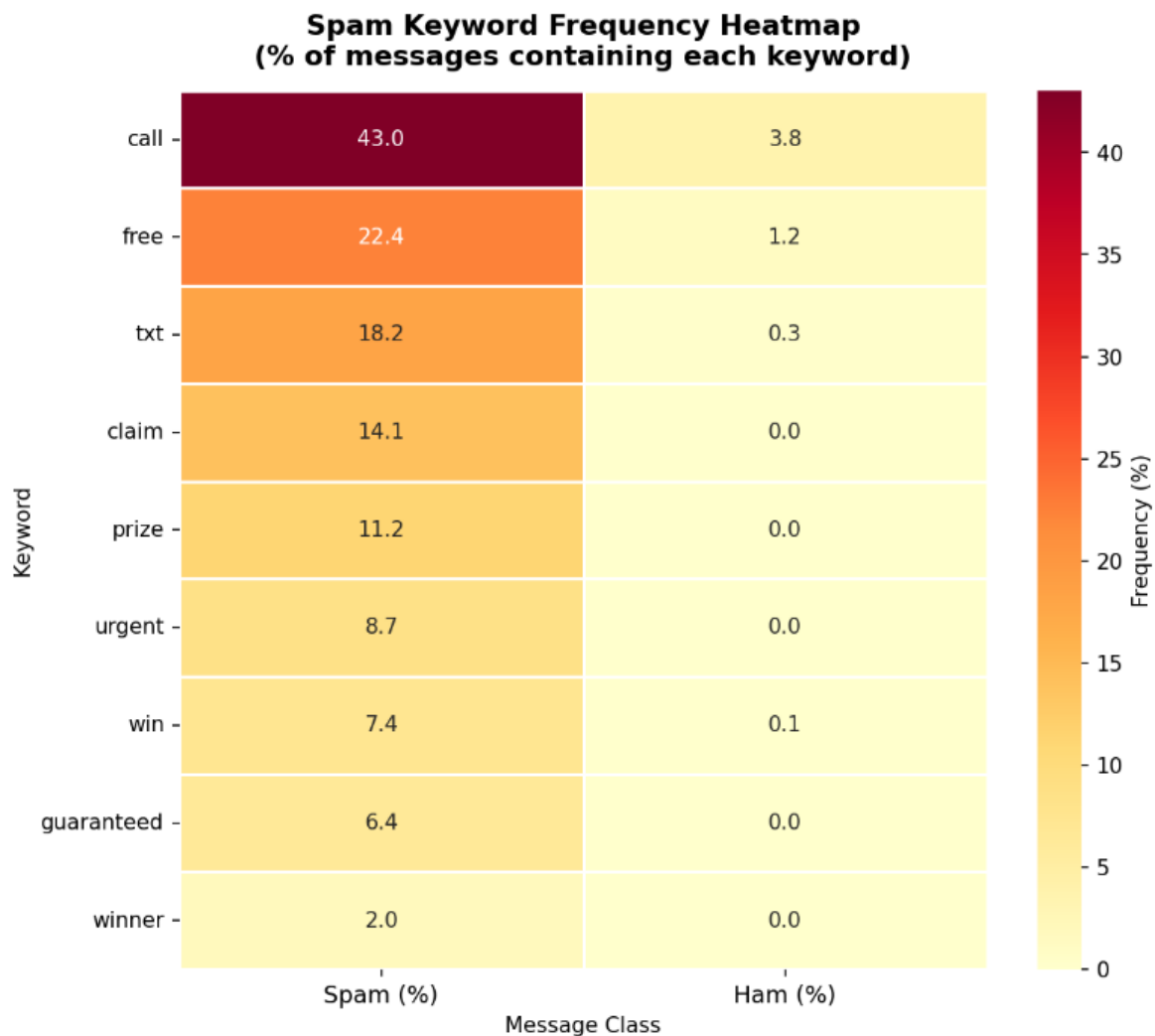


Figure 4.3.1: Spam keyword frequency heatmap showing spam vs ham

For the malware dataset, features with variance below 0.01 were removed, leaving 39 features. These were then normalized using StandardScaler so that each feature had a mean of zero and a standard deviation of one.

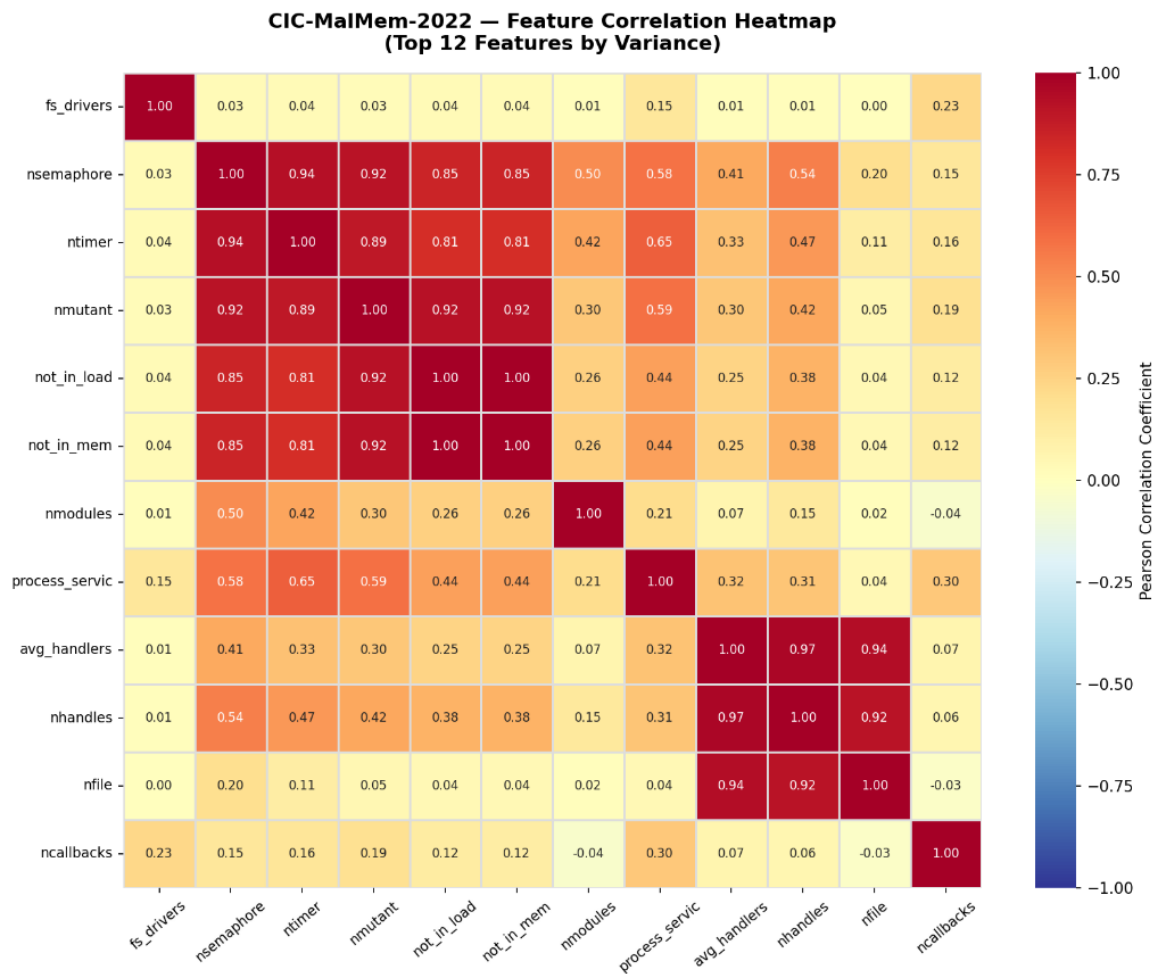


Figure 4.3.2: CIC-MalMem-2022 showing feature correlation heatmap

4.4 TF-IDF Vectorization

TF-IDF vectorisation was applied to convert the cleaned text into numerical vectors that machine learning models can work with. The vectorizer was configured with a vocabulary cap of 500 words for SMS data and 1,000 for email data, extracting both unigrams and bigrams [ngram_range=(1,2)], with English stopwords excluded. TF-IDF was chosen instead of raw word counts because it penalises words that appear across nearly every document, which allows distinctive spam-related terms to stand out more clearly in the feature space.

4.5 Train/Test Split

All datasets were divided into 80% training and 20% testing subsets using scikit-learn's train_test_split with stratified sampling. Stratification ensures that the spam-to-ham and malware-to-benign ratios in both subsets match the original dataset distribution, which is important for producing reliable precision, recall, and F1 score measurements.

5.0 Data Analysis

Before model training, exploratory data analysis was conducted on all four datasets to understand class distributions, feature characteristics, and preprocessing needs that would directly inform model design decisions.

SMS Spam Collection: The UCI SMS dataset contains 5,574 messages with a significant class imbalance, consisting of approximately 13% spam and 87% legitimate (ham). This imbalance is important because a naive classifier that always predicts "ham" would still achieve 87% accuracy without learning anything useful. Stratified sampling during the 80/20 train/test split was applied specifically to preserve this ratio and ensure evaluation metrics reflect real detection performance rather than class frequency bias. Message length analysis showed that spam messages tend to be notably longer than legitimate SMS messages, which motivated the inclusion of character count and word count as engineered features alongside TF-IDF.

Enron Email Dataset: The Enron corpus (33,716 labelled emails) provided a more balanced class distribution compared to the SMS dataset, with roughly equal proportions of spam and legitimate email. Including emails alongside SMS messages was intentional because the combined training corpus of approximately 39,290 samples exposed the classifiers to both short, keyword-heavy SMS spam and longer phishing-style emails, improving generalisation. The spam keyword heatmap confirmed that terms such as "free", "win", "claim", and "prize" appear overwhelmingly in spam samples, validating their inclusion as explicit keyword count features.

CIC-MalMem-2022 Dataset: The malware dataset contains 58,596 Windows process memory samples across four categories: Benign, Ransomware, Spyware, and Trojan. Initial inspection revealed 55 numerical features per sample. Variance analysis identified 16 features with variance below 0.01, which were removed as they carry negligible discriminative information, leaving 39 features for model training. The feature correlation heatmap showed several clusters of highly correlated memory features, which informed the dimensionality reduction applied before clustering. PCA was applied to the 39 scaled features, and the scree plot confirmed that 10 principal components capture 96.3% of total variance, providing a compact and near-lossless representation for K-Means. Five components were used for DBSCAN to reduce noise sensitivity in the density estimation step.

These observations, including class imbalance in SMS data, vocabulary patterns across spam types, and feature redundancy in memory data, directly shaped the preprocessing decisions and model configurations described in the following section.

The cross-validation standard deviations (± 0.0011 for Random Forest, ± 0.0003 for SVM) confirmed that these patterns in the data are stable across different splits, meaning the findings from exploratory analysis translated reliably into model generalisation rather than overfitting to one particular subset.

6.0 Machine Learning Model Selection

6.1 Selection Criteria

We selected models based on four considerations: compatibility with the data type (text features versus numerical memory features), ability to cover the three required method types, computational efficiency given the dataset sizes, and usefulness of the model output for a security application. In total, six models were chosen.

6.2 Classification Models

Random Forest was selected as the primary spam classifier. It builds 100 decision trees on random feature subsets and combines their predictions, which reduces overfitting (Breiman 2001) and handles mixed feature types well. It also produces feature importance scores, which helped us understand which inputs were most predictive. Parameters: `n_estimators=100`, `max_depth=15`, `random_state=42`. XGBoost was considered but excluded as scikit-learn's Random Forest provided comparable accuracy with simpler hyperparameter management for this project scope.

Naive Bayes (MultinomialNB) was included as a secondary classifier and comparison baseline. It operates directly on word frequency features and is fast and easy to interpret. Despite its simplifying assumption that features are independent of each other, it consistently performs well on text classification. Parameter: `alpha=1.0` (Metsis & Paliouras 2006).

Support Vector Machine (SVM) with RBF kernel was used for malware detection on the CIC-MalMem-2022 numerical features. SVM finds the maximum-margin boundary between classes (Cortes & Vapnik 1995), which gives it strong generalization on high-dimensional, scaled data. The RBF kernel allows it to model non-linear decision boundaries between malware categories. Because SVM training scales quadratically with sample size, training was done on a 20,000-sample stratified subset. Parameters: `kernel='rbf'`, `C=1.0`, `gamma='scale'`. K-Nearest Neighbours was considered but scales poorly with the 58,596-sample CIC-MalMem dataset; SVM on a 20,000-sample subset was more computationally tractable.

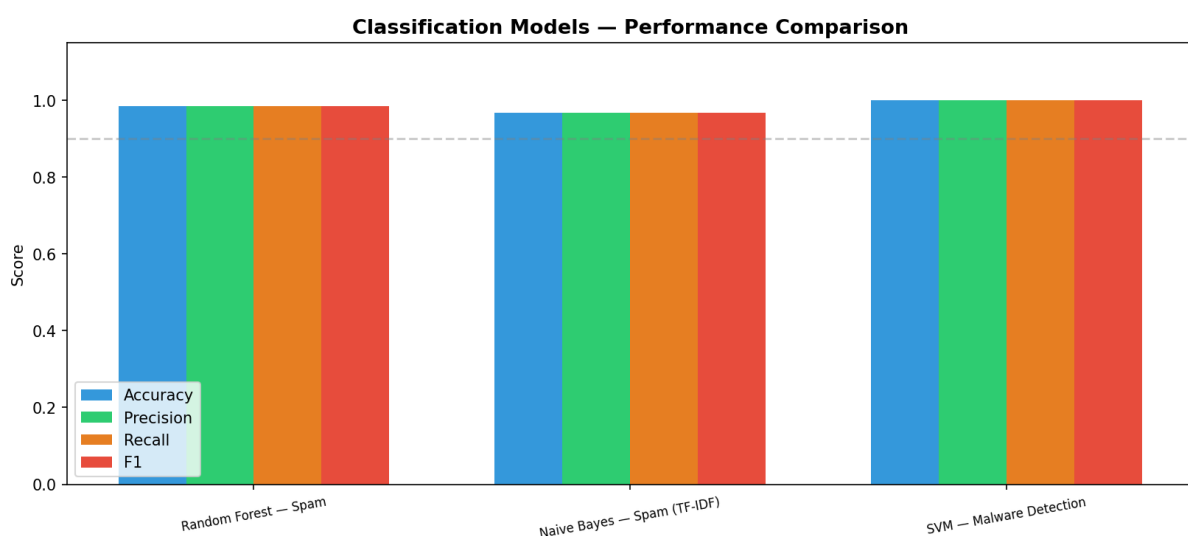


Figure 6.2: Classification model performance comparison

6.3 Regression Model

Logistic Regression was included as our regression-type method. While it is commonly described as a classifier, the mechanism behind it is regression-based: it models the log-odds of class membership using a linear equation, then passes the result through a sigmoid function to produce a continuous probability between 0 and 1. This is what qualifies it as a regression method in our pipeline. Instead of simply outputting "spam" or "not spam", the model gives a probability such as 0.91 (very likely spam) or 0.28 (probably legitimate), which is more useful for users who need to decide how to act on a borderline message. Its coefficients also reveal which words most strongly push the spam probability in either direction. Parameters: `C=1.0`, `max_iter=1000`, `solver='lbfgs'`.

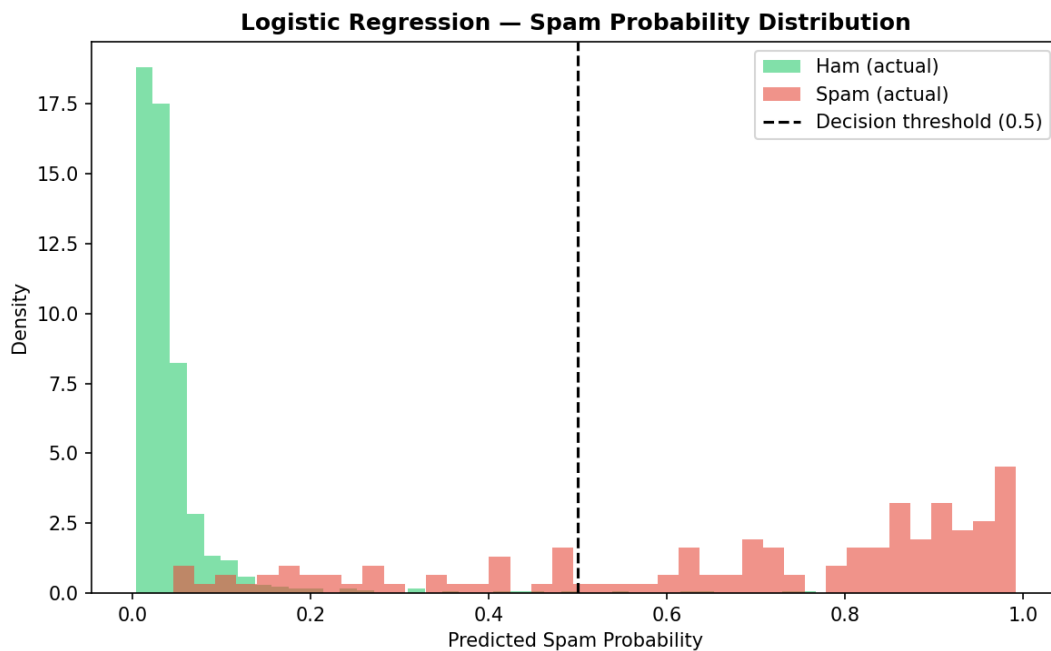


Figure 6.3: Logistic Regression showing spam probability distribution

6.4 Clustering Models

K-Means was applied to 15,000 sampled CIC-MalMem-2022 records after reducing dimensionality to 10 PCA components, which capture 96.3% of the total variance. The optimal number of clusters $K=6$ was selected based on the elbow method and silhouette score analysis across $K=2$ to $K=8$, with $K=6$ giving the highest silhouette score of 0.5668.

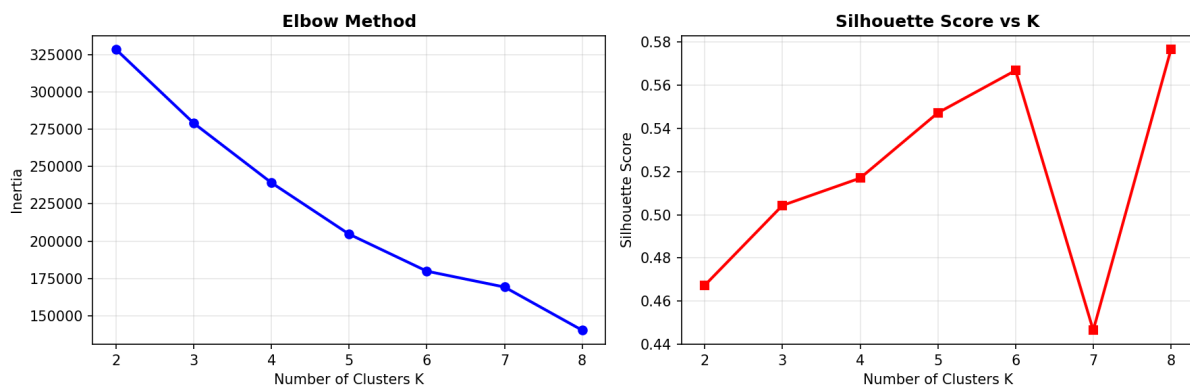


Figure 6.4.1: Elbow method and silhouette score analysis

DBSCAN was used as the second clustering method. Unlike K-Means, it does not assign every point to a cluster, so samples in low-density regions are labelled as noise (label -1), making it useful for flagging anomalous memory behavior that does not match any known malware family. It also does not require the number of clusters to be specified in advance (Ester et al. 1996). Parameters: $\text{eps}=0.8$, $\text{min_samples}=15$, applied to 5 PCA components. Isolation Forest was considered as an alternative anomaly detector but DBSCAN's density-based noise labelling aligns more directly with identifying behavioural outliers in memory data.

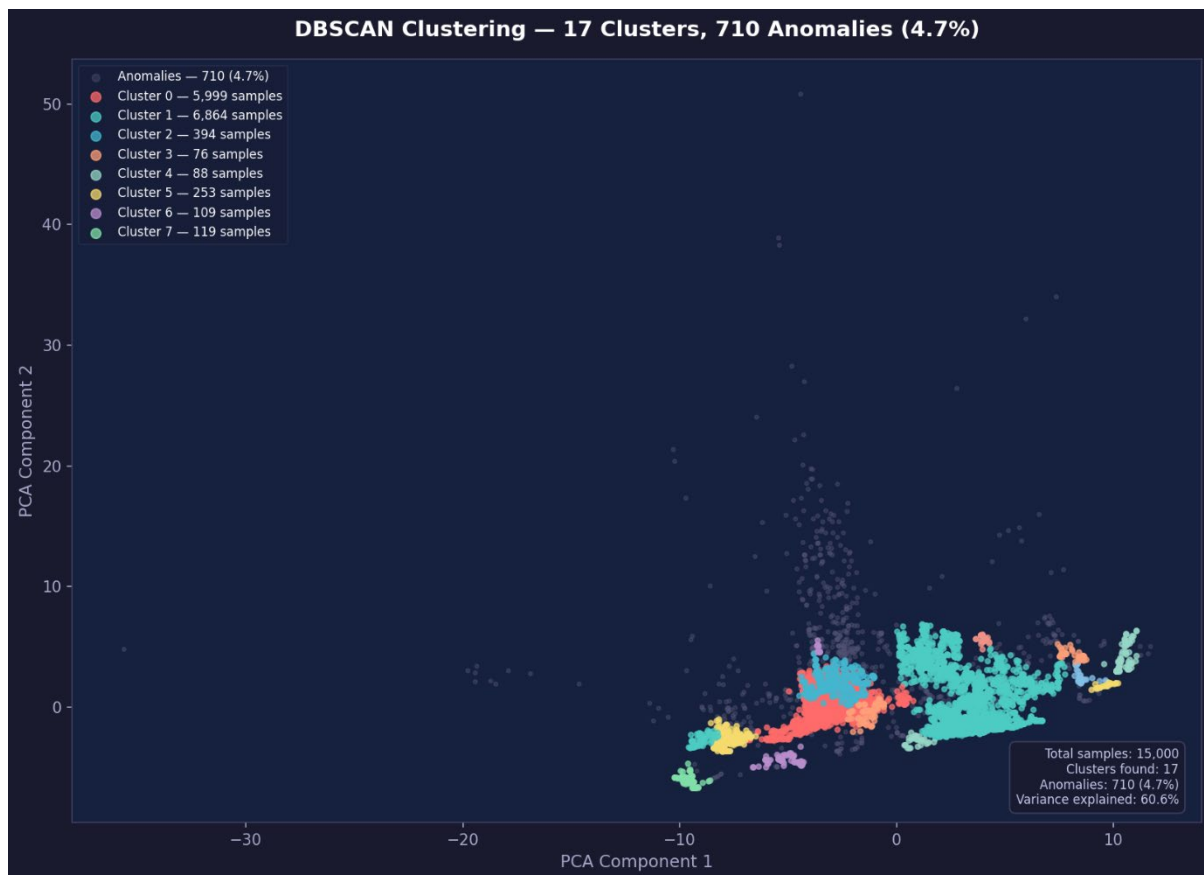


Figure 6.4.2: DBSCAN clustering results and anomaly points

7.0 Implementation

7.1 Technical Implementation

The NTCyber AI pipeline is organized into two main folders: preprocessing/ for all data preparation scripts, and models/ for training and evaluation. Two master scripts allow the full pipeline to be reproduced with a single command: 00_run_all_preprocessing.py runs all four preprocessing scripts in order, and 08_run_all_models.py runs the classification, clustering, and regression training scripts sequentially.

Libraries used: pandas and numpy for data handling and numerical computation; scikit-learn for TF-IDF vectorization (Pedregosa et al. 2011), all model training, cross-validation, and evaluation metrics; matplotlib and seaborn for generating visualization charts.

We ran into three technical challenges during development. First, the CIC-MalMem-2022 category encoding broke because the label encoder was accidentally applied to the process name column, which had 28,346 unique values, instead of the category name column, which had four. This was fixed by explicitly targeting category_name with a cardinality guard. Second, SVM training on all 58,596 samples was too slow due to its quadratic time complexity, so a 20,000-sample stratified subset was used with a fixed random seed for reproducibility. Third, Matplotlib popup windows caused the pipeline to freeze when scripts were run as subprocesses. This was resolved by setting the Agg non-interactive backend at the start of each script and replacing all plt.show() calls with plt.close().

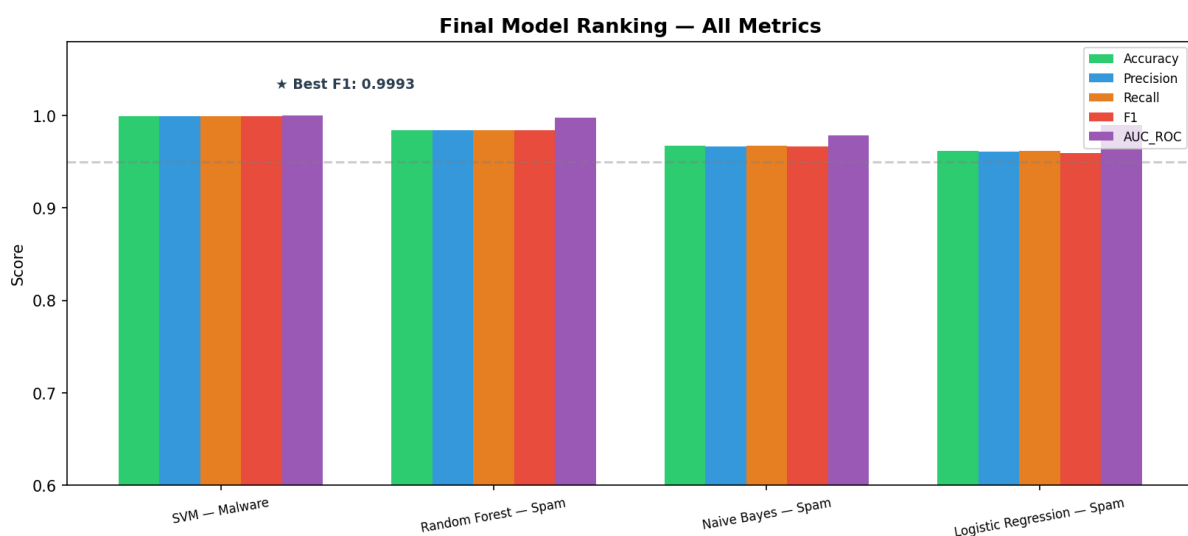


Figure 7.1: Final model ranking summary

7.2 Evaluation Results

All classifiers were evaluated on a held-out 20% test set, and five-fold stratified cross-validation was run to confirm that the results were stable across different splits.

Model	Accuracy	Precision	Recall	F1 Score	CV F1 (Mean $\pm$ Std)	AUC-ROC
Random Forest (Spam)	98.39%	98.39%	98.39%	98.39%	0.9831 $\pm$ 0.0011	0.9978
Naive Bayes (Spam)	96.71%	96.63%	96.71%	96.62%	0.9646 $\pm$ 0.0024	0.9787
SVM (Malware Detection)	99.92%	99.93%	99.92%	99.93%	0.9989 $\pm$ 0.0003	1.0000
Logistic Regression (Spam)	96.13%	95.05%	73.28%	82.76%	0.9621 $\pm$ 0.0074	0.9899

Five-fold stratified cross-validation was run on all classifiers to confirm that the test set results were not a product of a fortunate split. The results were highly stable across all models. Random Forest achieved a mean cross-validation F1 of 0.9831 with a standard deviation of just ± 0.0011 , indicating consistent performance regardless of which 20% of the data was held out. SVM was the most stable of all models, with CV F1 of 0.9989 ± 0.0003 , reflecting effectively zero variance across folds, which reflects how linearly separable the malware memory features are in the scaled feature space. Naive Bayes was similarly consistent at 0.9646 ± 0.0024 . Logistic Regression showed the widest spread at ± 0.0074 , still a narrow band, though its CV F1 of 0.9621 is notably higher than its test set F1 of 0.8276.

This gap arises because the CV metric is computed as a weighted average across both spam and ham classes, while the test set F1 reported above is calculated specifically on the spam class, where the model's conservative recall of 73.28% brings the binary score down. This distinction is important for interpreting the two numbers correctly rather than treating them as contradictory.

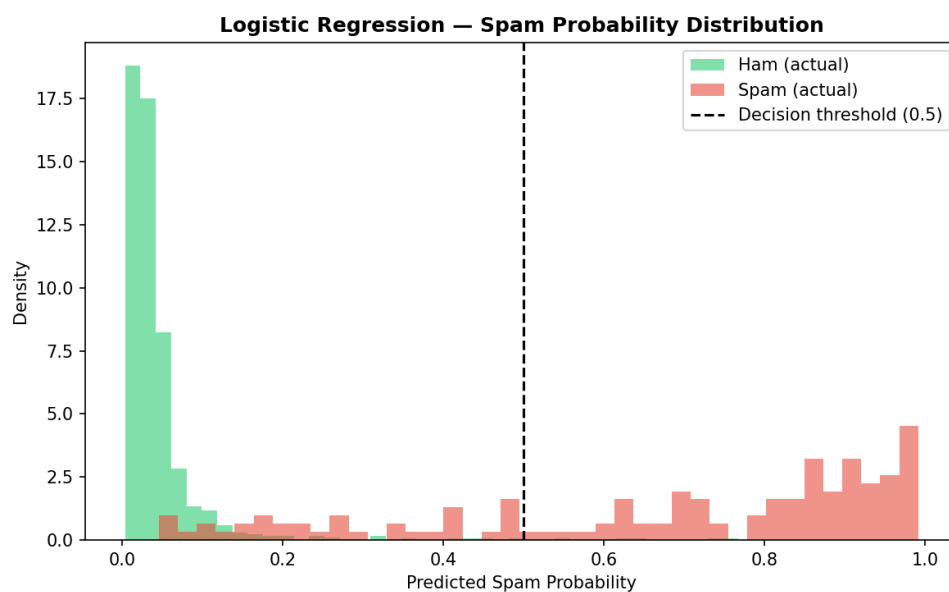


Figure 7.2.1: Mean spam probability for spam class vs ham class

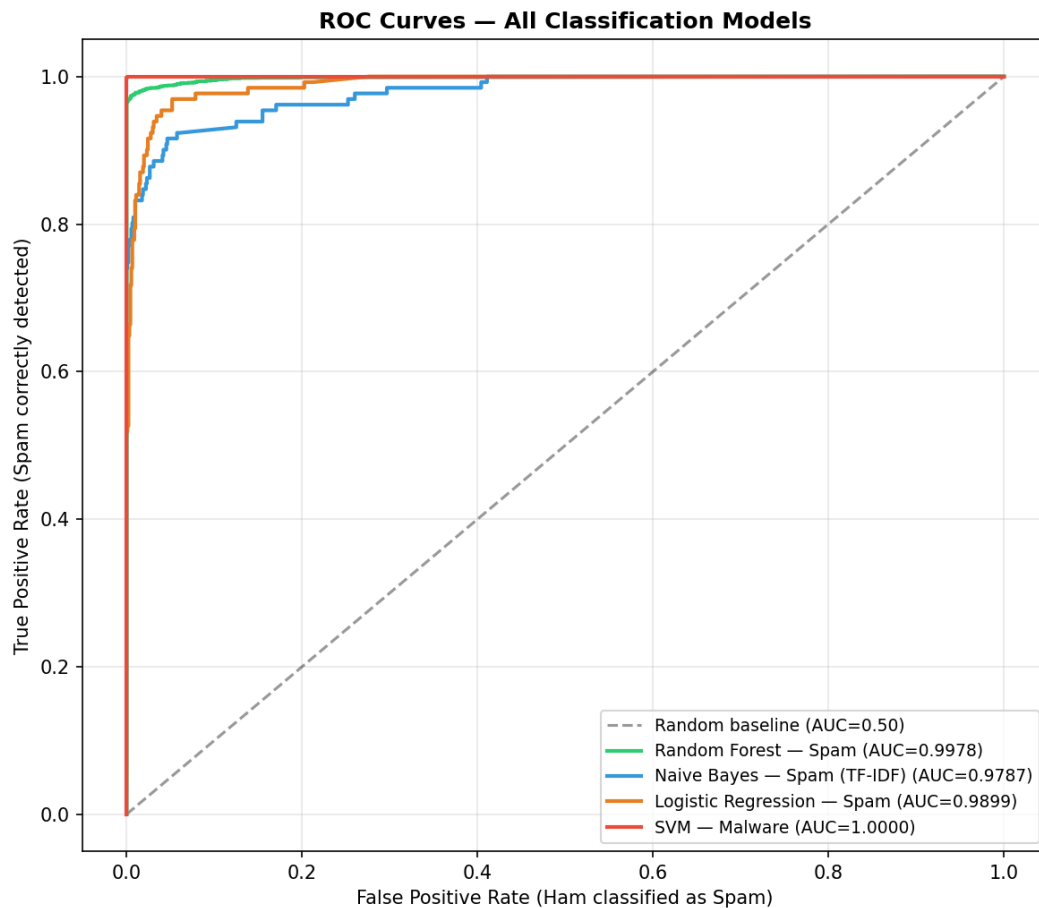


Figure 7.2.2: ROC curves showing all classification models

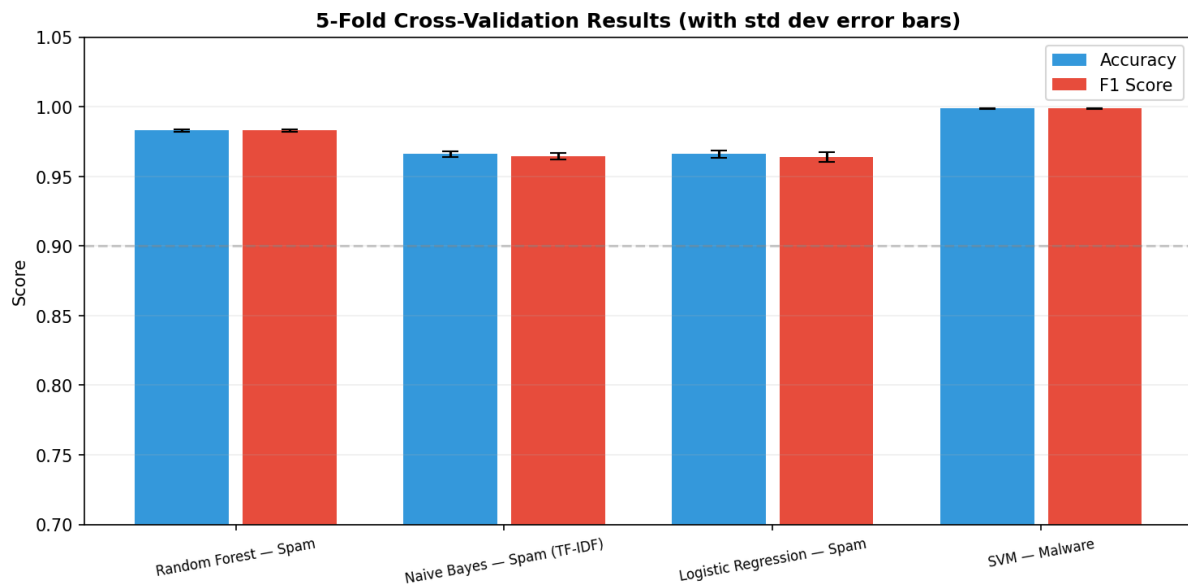


Figure 7.2.3: Cross-validation comparison

Random Forest was the best overall spam classifier at 98.39% accuracy. Its false positive rate of 0.95% and false negative rate of 2.38% gave the best balance between missing spam and wrongly blocking legitimate messages, which is an important consideration for general users.

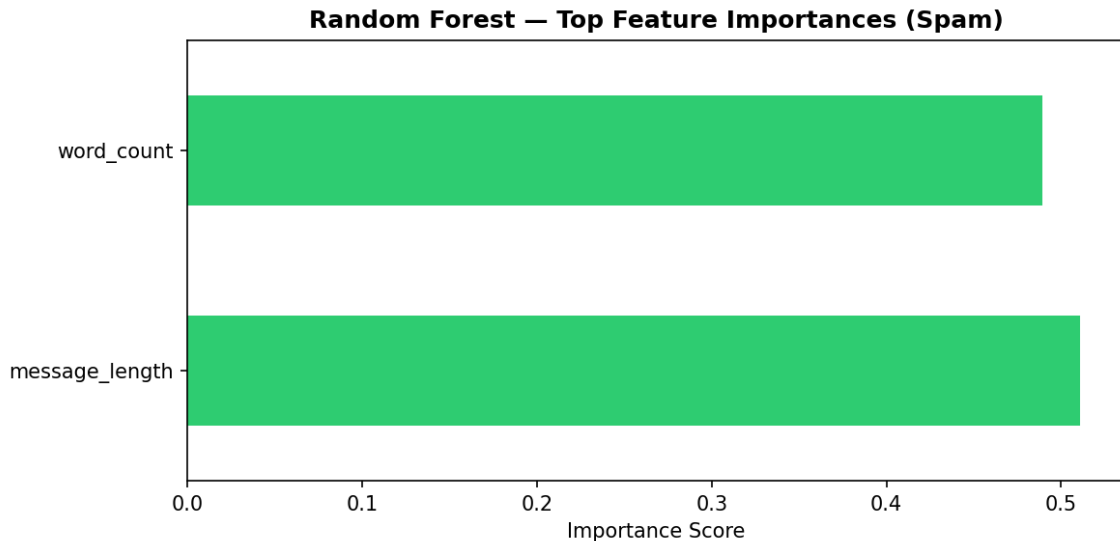


Figure 7.2.4: Random Forest showing top feature importances

SVM achieved near-perfect results on the malware dataset ($F1 = 0.9993$, $AUC = 1.0000$), which reflects how discriminative Windows process memory features are at separating malware from benign processes.

Logistic Regression had a recall of 73.28%, meaning roughly one in four spam messages was classified as legitimate. While this appears weak compared to Random Forest, the trade-off is deliberate and context dependent. Logistic Regression produced the lowest false positive rate of all classifiers at 0.55%, meaning legitimate messages are almost never wrongly blocked. For office workers and small business owners in the intended user group, a false positive, where a genuine client email flagged as spam and moved to junk, can have immediate professional consequences, such as a missed deadline or an unanswered business enquiry. In contrast, a missed spam message is an inconvenience that the user can still identify manually. This makes a conservative, high-precision model appropriate as a secondary layer. Additionally, Logistic Regression's continuous probability output, for example 0.91 versus 0.28 gives users a graduated confidence scores rather than a binary verdict, which is more informative for borderline cases. If higher recall is needed in future, the classification threshold could be lowered from the default 0.5 to approximately 0.35, which would recover more spam at a modest cost to precision.

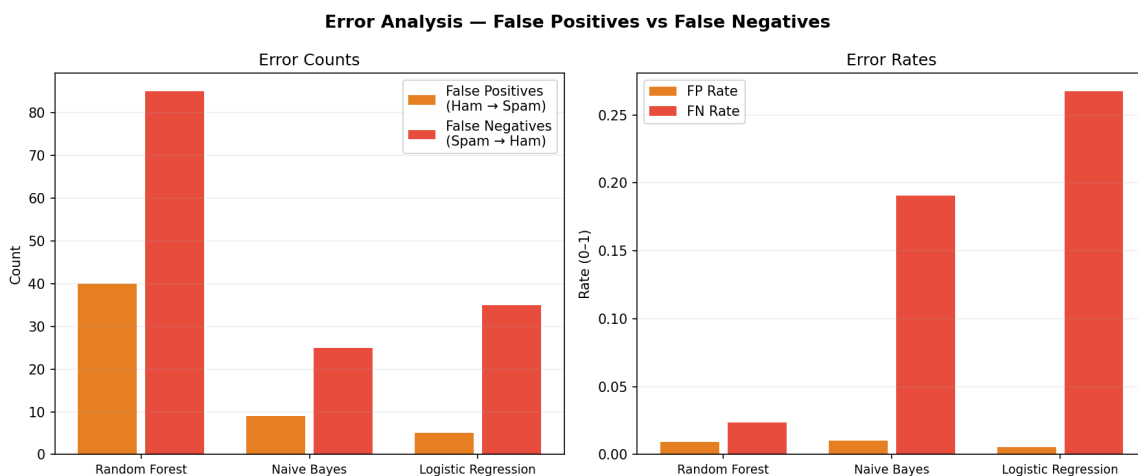


Figure 7.2.5: Error analysis showing false positive vs false negative rates

K-Means (K=6, silhouette = 0.5668) produced six clusters from 15,000 CIC-MalMem-2022 samples. Since the dataset contains four ground-truth categories - Benign, Ransomware, Spyware, and Trojan - the six-cluster solution suggests that at least two malware families are internally heterogeneous enough to split into sub-groups under unsupervised conditions. Cross-referencing cluster membership with ground-truth labels showed that benign samples concentrated heavily in one or two clusters, while malware samples spread across the remaining clusters with some overlap between Spyware and Trojan groups. This overlap is consistent with the known behavioural similarity between these two categories, both of which involve covert data access without the file encryption characteristic of Ransomware. The moderate silhouette score of 0.5668 reflects this partial overlap rather than a failure of the method, indicating that malware families leave distinguishable but not perfectly separable patterns in process memory.

DBSCAN identified 17 clusters and flagged 710 samples (4.7%) as noise points, meaning that these memory snapshots that did not belong to any dense region in the feature space. These anomalous samples are the most operationally significant output of the clustering analysis. Unlike K-Means, which forces every point into a cluster, DBSCAN's noise label signals that these samples behave differently from all known malware families in the training data. In a deployed system, such samples would be escalated for manual review as potential zero-day or obfuscated threats. The fact that DBSCAN recovered 17 distinct clusters from four labelled categories also suggests meaningful sub-variants within each malware family, which could be a direction for more granular threat classification in future work.

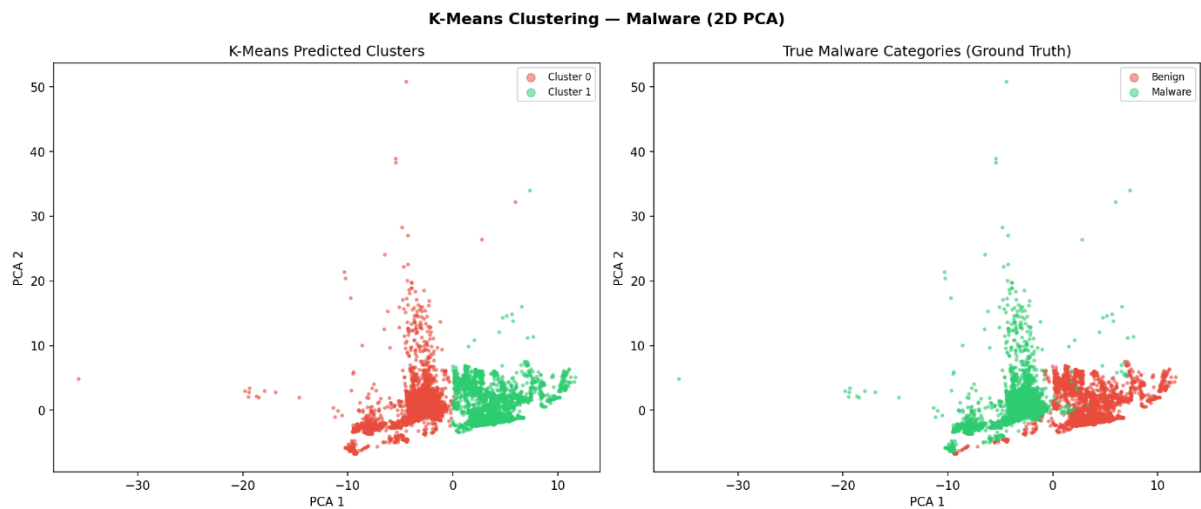


Figure 7.2.6: K-Means cluster visualisation

8.0 Conclusion

This project built and evaluated NTCyber AI, a machine learning pipeline for detecting both spam messages and malware. Six models were trained across three method types using four real-world datasets, with strong performance across all tasks.

For malware detection, SVM reached near-perfect accuracy ($F1 = 0.9993$, $AUC = 1.0000$), showing that process memory features are highly effective for separating malicious and benign processes. For spam detection, Random Forest performed best at 98.39% accuracy with a false positive rate of just 0.95%. Since wrongly blocking a legitimate email is more immediately disruptive to users than occasionally letting spam through, Random Forest is the recommended model for deployment.

Logistic Regression added value through its continuous probability output, giving users a graduated spam score beyond a binary verdict. K-Means showed that malware families cluster naturally by memory behaviour, while DBSCAN flagged 710 anomalous samples that could represent novel or obfuscated threats not seen during training.

The main limitations of the current system are its English-only focus and the static nature of the trained models. Spam patterns evolve over time, so periodic retraining on new data would be necessary for long-term real-world use. For future improvements, transformer-based models such as BERT could improve contextual understanding for spam detection, and incorporating dynamic behavioural features such as system call sequences alongside the existing memory features would strengthen the malware detection component. The pipeline and trained models built in this project provide a solid foundation for both those future extensions and for full-stack integration in Assignment 3.

Taken together, the six models address different threat profiles and user needs. Random Forest serves as the primary spam filter for general users who need reliable, low-disruption detection. Logistic Regression provides a probability score that supports more informed decision-making for borderline messages. SVM delivers near-perfect malware classification on structured memory data, while DBSCAN provides a mechanism for surfacing unknown threats that fall outside any trained category. This layered approach, which combine high-accuracy classifiers with anomaly-aware clustering, reflects how real-world security systems handle the fact that not all threats can be anticipated in advance.

9.0 Bibliography

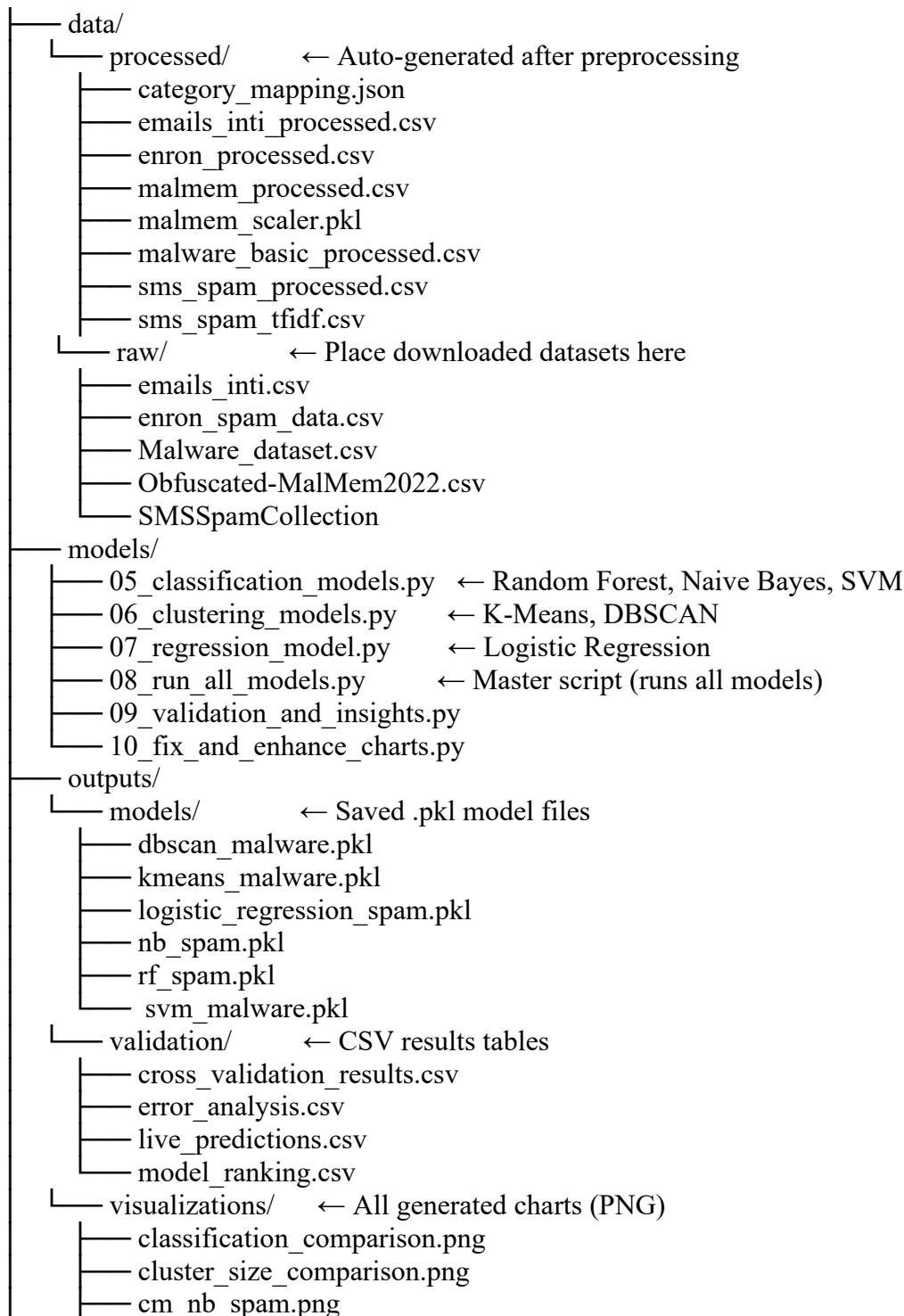
1. Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering. *Proceedings of the 11th ACM Symposium on Document Engineering - DocEng '11*. <https://doi.org/10.1145/2034691.2034742>
2. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), pp.5–32. <https://doi.org/10.1023/A:1010933404324>
3. www.unb.ca. (n.d.). *Malware Memory Analysis | Datasets | Canadian Institute for Cybersecurity | UNB*. [online] Available at: <https://www.unb.ca/cic/datasets/malmem-2022.html>.
4. Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), pp.273–297. doi: <https://doi.org/10.1007/BF00994018>.
5. Ester, M., Kriegel, H.-P., Sander, J. and Xu, X. (1996). *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*. [online] Available at: <https://cdn.aaii.org/KDD/1996/KDD96-037.pdf>.
6. Metsis, V. and Paliouras, G. (n.d.). *Spam Filtering with Naive Bayes -Which Naive Bayes? **. [online] Available at: https://www2.aueb.gr/users/ion/docs/ceas2006_paper.pdf.
7. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, [online] 12(85), pp.2825–2830. Available at: <https://jmlr.org/papers/v12/pedregosa11a.html>.
8. Wiechmann, M. (2021). *Enron Spam Data*. [online] Kaggle.com. Available at: <https://www.kaggle.com/datasets/marcelwiechmann/enron-spam-data>.

10.0 Appendix

10.1 Appendix A: Project Structure

The NTCyber project is organized into the following directory structure. All preprocessing scripts are located under the preprocessing/ folder and must be run before the model training scripts in the models/ folder. Processed datasets and trained model files are automatically saved to the outputs/ directory.

COS30049---Assignment-G1/



- cm_rf_spam.png
- cm_svm_malware.png
- cross_validation_comparison.png
- dbscan_clusters_fixed.png
- dbscan_clusters.png
- emails_inti_analysis.png
- enron_analysis.png
- error_analysis.png
- final_model_ranking.png
- kmeans_clusters.png
- kmeans_elbow.png
- lc_lr_spam.png
- lc_nb_spam.png
- lc_rf_spam.png
- lc_svm_malware.png
- lr_coefficients.png
- lr_confusion_matrix.png
- lr_probability_distribution.png
- lr_roc_curve.png
- malmem_analysis_fixed.png
- malmem_correlation_heatmap.png
- malmem_feature_boxplot.png
- malware_basic_analysis.png
- pca_scree_plot.png
- rf_feature_importance.png
- roc_all_models.png
- sms_spam_analysis.png
- spam_keyword_heatmap.png
- all_results_combined.csv
- classification_results.csv
- clustering_results.csv
- regression_results.csv
- preprocessing/
 - 00_run_all_preprocessing.py ← Master script (runs all preprocessing)
 - 01_preprocess_sms_spam.py
 - 02_preprocess_malmem.py
 - 03_preprocess_enron.py
 - 04_preprocess_basic_datasets.py
 - README_preprocessing.md
- .gitignore
- README.md

10.2 Appendix B: Project Environment Setup

The following steps set up the Python environment needed to run the NTCyber pipeline. Open Anaconda Prompt (not Git Bash) and run the commands below.

Step 1: Create and activate the conda environment

```
conda create -n spam_malware python=3.10
```

When prompted, type y and press Enter. Then activate the environment:

```
conda activate spam_malware
```

Step 2: Install required packages

```
pip install pandas numpy scikit-learn matplotlib seaborn
```

To verify the installation ran correctly:

```
python -c "import pandas, numpy, sklearn, matplotlib, seaborn; print('All packages OK')"
```

Step 3: Run data preprocessing

Navigate to the project root and run the preprocessing master script. This cleans and prepares all four datasets automatically:

```
cd preprocessing
```

```
python 00_run_all_preprocessing.py
```

Step 4: Train all models

Run the model training master script from the models/ folder. SVM training takes approximately 2 to 3 minutes on its 20,000-sample subset, which is normal.

```
cd ../models
```

```
python 08_run_all_models.py
```

Step 5: Run validation and generate charts

These two scripts generate all evaluation metrics, learning curves, ROC curves, and report-quality charts. The second script takes approximately 5 to 10 minutes due to cross-validation computations.

```
python 09_validation_and_insights.py
```

```
python 10_fix_and_enhance_charts.py
```

All trained models are saved as .pkl files in outputs/models/. Charts are saved to outputs/visualizations/ and validation results to outputs/validation/. See README.md for prediction usage examples.

10.3 Appendix C: Meeting Minutes

Meeting Minutes 4

Team name: Session01-Group1

Meeting No: 4

Date: 27 April 2026

Location: Online (Discord)

Attending: Tee Ren Hang, Ng Yong Hin

Apologies: None

Meeting start time: 8:00 PM

Matters arising from Previous minutes: Yes

Issues raised from previous minutes	Discussions	Outcome (Resolved?)
Assignment 1 submission	Assignment 1 submitted on Canvas on 24 Apr by Tee Ren Hang	Resolved
Report and prototype completion	All sections, Figma screenshots, and Gantt chart inserted before submission	Resolved

Confirmation of minutes from last meeting: Yes

Outcome of meeting:

Issue	Discussion in brief	Outcome	Action: Name and Timeline
Assignment 2 kickoff and task planning	Both members reviewed the Assignment 2 specification. Key deliverables noted: report up to 3,000 words, processed datasets, README, source code, meeting minutes, and contribution form. Both agreed to source additional datasets beyond the basic dataset provided.	Assignment 2 scope confirmed. Both to identify and download additional datasets before next meeting.	Both - By 30 Apr 2026
Dataset identification and selection	Ng Yong Hin proposed three additional datasets: UCI SMS Spam Collection, Enron Email Spam (Kaggle), and CIC-MalMem-2022. Both agreed this suit the spam and malware detection objectives.	Three datasets confirmed. Ng Yong Hin to download and inspect all datasets.	Ng Yong Hin - By 30 Apr 2026
ML model types and approach	Both reviewed the requirement for at least two ML method types. Six models agreed: Random Forest and Naive Bayes for spam classification, SVM for malware classification, K-Means and DBSCAN for clustering, and Logistic Regression for regression.	Six-model plan confirmed across three method types. Ng Yong Hin to begin preprocessing scripts.	Ng Yong Hin - By 4 May 2026
Report responsibilities	Report sections divided: Tee Ren Hang to write Introduction, Problem Framing, Data Collection, Conclusion, and Bibliography. Ng Yong Hin to write Data Processing, Data Analysis, Model Selection, Implementation, and Evaluation.	Report writing responsibilities confirmed. Tee Ren Hang to begin drafting his sections.	Tee Ren Hang - By 4 May 2026

Actions in brief: Ng Yong Hin to begin downloading datasets and setting up the Python preprocessing pipeline. Tee Ren Hang to draft the Introduction, Problem Framing, and Data Collection sections of the Assignment 2 report.

Meeting closed at: 9:30 PM

Next meeting time, date and location: Meeting 5 - 4 May 2026, 8:00 PM, Online (Discord)

Meeting Minutes 5

Team name: Session01-Group1

Meeting No: 5

Date: 4 May 2026

Location: Online (Discord)

Attending: Tee Ren Hang, Ng Yong Hin

Apologies: None

Meeting start time: 8:00 PM

Matters arising from Previous minutes: Yes

Issues raised from previous minutes	Discussions	Outcome (Resolved?)
Dataset selection	UCI SMS Spam, Enron Email Spam, and CIC-MalMem-2022 datasets downloaded and inspected	Resolved
Report draft - Introduction, Problem Framing, and Data Collection	Sections drafted by Tee Ren Hang; to be reviewed at this meeting	Resolved

Confirmation of minutes from last meeting: Yes

Outcome of meeting:

Issue	Discussion in brief	Outcome	Action: Name and Timeline
Preprocessing pipeline progress	Ng Yong Hin presented completed preprocessing scripts. SMS and email data was cleaned, feature-engineered, and TF-IDF vectorised. CIC-MalMem-2022 had 16 low-variance features removed, leaving 39 features after StandardScaler normalisation. No missing values across all processed datasets.	Preprocessing pipeline complete. Processed datasets saved as CSV files ready for model training.	Ng Yong Hin - Completed 4 May 2026
Model training progress	Logistic Regression, Naive Bayes, and Random Forest trained on spam datasets. SVM on the full CIC-MalMem-2022 dataset was too slow, so a 20,000-sample subset was agreed. K-Means and DBSCAN scripts in progress.	Spam classifiers complete. SVM to use 20,000-sample subset. Clustering models to be done before Meeting 6.	Ng Yong Hin - By 11 May 2026
Report sections review	Tee Ren Hang presented drafted Introduction, Problem Framing, and Data Collection sections. Both confirmed content was accurate. Ng Yong Hin suggested adding more detail on the Enron dataset labelling challenge in the Data Collection section.	Three sections approved. Data Collection to be updated with Enron labelling challenge detail.	Tee Ren Hang - By 11 May 2026
Data analysis and visualizations	Both discussed the data analysis section and the charts to include in the report. Agreed list: class distribution charts, message length histograms, spam keyword heatmap, feature variance chart, PCA scree plot, elbow and silhouette chart, and model comparison chart. Ng Yong Hin to write Section 5.0 based on the analysis findings.	Visualisation list and Data Analysis section scope confirmed. Ng Yong Hin to generate and export all charts as PNG files.	Ng Yong Hin - By 11 May 2026

Actions in brief: Ng Yong Hin to complete clustering model training and generate all visualization charts. Tee Ren Hang to update the Data Collection section and begin drafting the Conclusion. Both to prepare a full report draft for review at Meeting 6.

Meeting closed at: 9:45 PM

Next meeting time, date and location: Meeting 6 - 11 May 2026, 8:00 PM, Online (Discord)

Meeting Minutes 6

Team name: Session01-Group1

Meeting No: 6

Date: 11 May 2026

Location: Online (Discord)

Attending: Tee Ren Hang, Ng Yong Hin

Apologies: None

Meeting start time: 8:00 PM

Matters arising from Previous minutes: See table below

Issues raised from previous minutes	Discussions	Outcome (Resolved?)
Preprocessing pipeline and model training	All preprocessing complete; spam classifiers and SVM trained and evaluated	Resolved
Report sections: Data Collection and Processing	Sections updated with Enron dataset challenge detail	Resolved
Visualisations	All charts generated and exported as PNG files	Resolved

Confirmation of minutes from last meeting: Minutes of Meeting 5 confirmed as accurate by both members.

Outcome of meeting:

Issue	Discussion in brief	Outcome	Action: Name and Timeline
Clustering model results and evaluation	Ng Yong Hin presented K-Means and DBSCAN results. K-Means with K=6 gave the best silhouette score. DBSCAN identified noise points as potential anomalous samples. Full model performance reviewed: SVM 99.92%, Random Forest 98.39%, Naive Bayes 96.71%, Logistic Regression 96.13%.	All six models complete. Evaluation results table finalized. Ng Yong Hin to write up the Implementation and Evaluation sections.	Ng Yong Hin - By 14 May 2026
Full report draft review	Both reviewed the complete report draft. Minor edits agreed: adding cross-validation result table, clarifying Logistic Regression recall trade-off, and adding a learning curve summary.	Report draft approved with minor edits. Final version to be completed by 14 May 2026.	Both - By 14 May 2026
ZIP package and README preparation	Ng Yong Hin to prepare the submission ZIP with processed CSV datasets, source code scripts, trained model pickle files, and a README covering conda setup, preprocessing, model training, and prediction usage.	ZIP file structure agreed. README to cover environment setup, preprocessing, training, and prediction steps.	Ng Yong Hin - By 20 May 2026
Submission plan and final checks	Tee Ren Hang outlined the submission package: report PDF, ZIP file, meeting minutes 4 to 7 as PDF, and contribution form as PDF. Final review to be conducted at Meeting 7 on 18 May.	Submission package structure confirmed. Final review and submission to be completed at Meeting 7.	Tee Ren Hang - By 23 May 2026, 11:59 PM

Actions in brief: Ng Yong Hin to finalize the report Implementation and Evaluation sections, prepare the ZIP package with all code and datasets, and complete the README. Tee Ren Hang to apply final report edits and prepare all documents for export. Both to bring all completed files to Meeting 7 for final review before submission.

Meeting closed at: 9:30 PM

Next meeting time, date and location: Meeting 7 - 18 May 2026, 8:00 PM, Online (Discord)

Meeting Minutes 7

Team name: Session01-Group1

Meeting No: 7

Date: 18 May 2026

Location: Online (Discord)

Attending: Tee Ren Hang, Ng Yong Hin

Apologies: None

Meeting start time: 8:00 PM

Matters arising from Previous minutes: See table below

Issues raised from previous minutes	Discussions	Outcome (Resolved?)
Report Implementation and Evaluation sections	Ng Yong Hin completed the Implementation and Evaluation sections in the report before this meeting	Resolved
ZIP package and README preparation	ZIP file containing all code, datasets, and trained models prepared; README completed in Markdown format	Resolved
Report final edits	Tee Ren Hang applied all agreed minor edits from Meeting 6 to the report draft	Resolved

Confirmation of minutes from last meeting: Minutes of Meeting 6 confirmed as accurate by both members.

Outcome of meeting:

Issue	Discussion in brief	Outcome	Action: Name and Timeline
Final report review and proofreading	Both reviewed the full report. Tee Ren Hang checked his sections for accuracy and Harvard citation style. Ng Yong Hin verified technical sections for accuracy and correct figure references. Minor typos corrected. Report confirmed complete and ready for PDF export.	Final report approved. Tee Ren Hang to export as PDF before submission.	Tee Ren Hang - By 20 May 2026
ZIP package and README verification	Ng Yong Hin walked through the ZIP contents: processed CSV datasets, source code scripts, trained model .pkl files, and README.md. Tee Ren Hang verified the README instructions are clear and cover setup, preprocessing, training, and prediction.	ZIP package verified and complete. README confirmed clear and accurate.	Ng Yong Hin - Completed 18 May 2026
Meeting minutes and contribution form finalisation	Both reviewed Meeting Minutes 4 through 7 for accuracy. Tee Ren Hang confirmed the Contribution Form reflects the correct Assignment 2 title and contributions. Both agreed on the 50% contribution split and ratings before signing.	All four meeting minutes confirmed. Contribution Form signed by both members.	Tee Ren Hang - By 20 May 2026
Final submission on Canvas	Tee Ren Hang to submit all Assignment 2 files on Canvas on behalf of the group. Files checklist confirmed: report PDF, ZIP file, meeting minutes 4 to 7 as PDF, and contribution form as PDF. All files verified to follow the correct naming convention.	Submission checklist completed. Tee Ren Hang to submit all A2 files on Canvas by 23 May 2026, 11:59 PM.	Tee Ren Hang - By 23 May 2026, 11:59 PM

Actions in brief: Tee Ren Hang to export the report and all meeting minutes as PDF and submit the complete Assignment 2 package on Canvas by 23 May 2026, 11:59 PM. Ng Yong Hin to be available for any last-minute corrections before submission.

Meeting closed at: 9:30 PM

Next meeting time, date and location: N/A - Assignment 2 submission deadline: 23 May 2026, 11:59 PM

10.4 Appendix D: Contribution Form

Group Assessment Contribution Form

Course: COS30049-Computing Technology Innovation Project

Project/Assignment Title: AI-based Project for cybersecurity (AI4Cyber)

Group Number: 1

1. Group Members:

Please list down all members of the group, including yourself:

1. Name: Tee Ren Hang Student ID: 106214467

2. Name: Ng Yong Hin Student ID: 106214441

2. Team Task & Contribution Log:

To be completed and signed by all group members. This section documents the specific tasks each member was responsible for and provides evidence of their work. All members must reach a consensus on the described contributions

- Example: " Design: The user login and registration pages using Figma."
- Example: " Documentation: The API endpoints for user authentication."
- Example: " Set up the project repository on GitHub. "

Group Member	Tasks
Tee Ren Hang	• Writing: The Introduction (Section 1) of the Assignment 2 report, covering project motivation, cybersecurity threat context, and intended user group. • Writing: The Problem Framing section (Section 2), articulating limitations of rule-based detection systems and the justification for a machine learning approach. • Writing: The Data Collection section (Section 3), describing the four datasets used, their sources and rationale, and challenges encountered during collection. • Writing: The Conclusion section (Section 8), summarising key findings across all six machine learning models and identifying future work directions. • Writing: The Bibliography (Section 9) in Harvard citation style, referencing all datasets, algorithms, and libraries cited in the report. • Documentation: Meeting Minutes 4, 5, 6, and 7 covering the full Assignment 2 development period. • Documentation: The Group Assessment Contribution Form for Assignment 2, ensuring both members reviewed and signed before submission. • Submission: Final assembly and Canvas submission of the complete Assignment 2 package on behalf of the group by 23 May 2026.

Ng Yong Hin	• Development: All data preprocessing scripts for four datasets which is UCI SMS Spam Collection, Enron Email Spam Dataset, CIC-MalMem-2022, and the course-provided basic dataset. • Development: Text cleaning pipeline including lowercasing, URL removal, special character removal, and whitespace normalisation for SMS and email datasets. • Development: Feature engineering for spam datasets, including message length, word count, capital letter ratio, and spam keyword flags. • Development: TF-IDF vectorisation (500 features for SMS, 1,000 for email) and StandardScaler normalisation for malware numerical features. • Development: Training and evaluation of all six machine learning models which is Random Forest, Naive Bayes, and SVM (classification); K-Means and DBSCAN (clustering); Logistic Regression (regression). • Development: Model evaluation pipeline including 80/20 train-test split, 5-fold stratified cross-validation, ROC-AUC analysis, confusion matrices, error analysis, and live prediction testing. • Writing: The Data Processing section (Section 4), Data Analysis section (Section 5), Model Selection section (Section 6), Implementation section (Section 7), and Evaluation Results section (Section 7.2) of the report. • Visualization: Generated all 31 chart PNG files saved to outputs/visualizations/, including classification comparisons, clustering plots, ROC curves, and feature analysis charts. • Deliverables: Prepared the submission ZIP file containing processed CSV datasets, Python source code scripts, trained model .pkl files, and README.md with environment setup and usage instructions.
-------------	--

3. Team Member Contributions:

As a group, discuss and evaluate the contributions of each team member in the table below. The final evaluations **must represent a consensus** agreed upon by the entire team.

Group Member	Contribution (100% in total)	Quality of Work (1-5)	Timeliness (1-5)	Collaboration (1-5)
Tee Ren Hang	50%	5	5	5
Ng Yong Hin	50%	5	5	5

Contribution and Grading Policy:

1. **Normal Grade:** Members contributing 33% or more will receive the full project mark.
2. **Penalty:** Members contributing less than 33% will incur a penalty. Their grade will be calculated as follows:
 - $(\text{Contribution Percentage}/33) \times \text{Group Mark}$

Example: If the total group mark is 15, and a member contributes 20%, their grade will be $(20/33) \times 15 = 9$

Member 1 Signature: RenHang

Member 2 Signature: YongHin

Date: 20th May 2026